Bayesian Machine Learning in Quantitative Finance

Chapter 3: SABR Model and Hamiltonian Monte Carlo

Based on Mongwe, Mbuvha & Marwala

Adapted for Lecture Slides

2026

Chapter Outline

Why Not Just Use Black–Scholes?

Intuition first. Black–Scholes (Black & Scholes, 1973) and Black's model (Black, 1976) are the classic tools for pricing vanilla options, but they both assume volatility is constant over the life of the option.

This is not what markets show us:

- For a fixed maturity, options with different strike prices trade at different *implied* volatilities – this is the **volatility smile / skew**.
- For a fixed strike, implied volatility also changes with time to maturity – the **term structure** of volatility.

Hedging a large book of options with a single constant-volatility number is not realistic, since a real portfolio pools options across many strikes and maturities together.

From Local Volatility to Stochastic Volatility

Attempt 1 – Local volatility (Dupire, 1994). Volatility is a deterministic function of the strike and current asset price. This fits today's smile perfectly, but:

- It predicts the skew should move in the *opposite* direction to the underlying's price – the reverse of what is typically observed (Hagan et al., 2002).
- Its delta-hedging formula contains a correction term that pushes the predicted skew the wrong way when the forward price changes.

Attempt 2 – Stochastic volatility (Hagan et al., 2002). Instead of making volatility a fixed function of price and strike, let volatility itself be a random process, correlated with the asset price process. This is the idea behind the **SABR** model (Stochastic Alpha Beta Rho).

Plain-language summary: SABR says "the asset's volatility is not a fixed number – it wiggles randomly over time, and it tends to move together with (or against) the asset price." This one extra layer of randomness lets the model reproduce the smile/skew shapes seen in the market, and gives a closed-form (approximate) pricing formula.

What This Chapter Does

- Calibrates the SABR model to **simulated** swaption data and to **real, ZAR (South African Rand) market-observed** swaption prices (as at 28 June 2024).
- Uses the **Bayesian inference framework**: instead of a single best-fit point estimate of the SABR parameters, we get a full *posterior distribution* over them, with naturally-arising confidence intervals on predictions (e.g. the fitted skew).
- Compares three Markov Chain Monte Carlo (MCMC) samplers for generating that posterior:
  1. Metropolis-Adjusted Langevin Algorithm (**MALA**)
  2. Hamiltonian Monte Carlo (**HMC**)
  3. Separable Shadow Hamiltonian Hybrid Monte Carlo (**S2HMC**)
- The book reports this is a **first-in-literature** use of S2HMC (together with HMC and MALA) to calibrate the SABR model.

Why should you care? This is the first chapter in this lecture series to actually build MCMC samplers from the ground up (Metropolis-Hastings $\rightarrow$ MALA $\rightarrow$ HMC $\rightarrow$ S2HMC). Every later chapter that uses HMC assumes you understood the mechanics taught here.

Roadmap for This Chapter

1. **3.2** The SABR model: its SDEs and its (approximate) closed-form implied volatility formula.
2. **3.3** The three MCMC samplers used to calibrate it: MALA, HMC, S2HMC – built up from scratch, with a toy running example.
3. **3.4** The numerical experiment setup: simulated and real ZAR swaption data, and the performance metrics used.
4. **3.5** Results: how well each method calibrates SABR and predicts on unseen data.
5. **3.6** Conclusions and what comes next.

SABR: The Big Picture

Intuition. SABR models two things moving randomly *together*:

- F_t: the forward price of the underlying asset (e.g. a forward interest rate, for swaptions).
- σ_t: the volatility of that forward price – which is *itself* a random process, not a constant.

The two random "shocks" driving F_t and σ_t are correlated by a parameter ρ. This correlation is what lets the model generate a downward- or upward-sloping skew, matching what is seen in real option markets.

The SABR SDEs

SABR model (Hagan et al., 2002)

$$dF_t = \sigma_t F_t^{\beta}\, dW_t \tag{1}$$
$$d\sigma_t = \sigma_t\, \nu\, dZ_t \tag{2}$$

Symbols:

- F_t – forward price process; σ_t – volatility process, with $\sigma_0 = \alpha$.
- ν – *volatility of volatility* ("Volvol"): how wildly the volatility itself fluctuates.
- $\beta \in [0, 1]$ – the *elasticity* parameter: controls how volatility scales with the level of F_t.
- W_t, Z_t – standard Brownian motions, correlated via $dW_t\, dZ_t = \rho\, dt$, with $\rho \in [-1, 1]$.

The model assumes the underlying is already a (local) martingale under some equivalent martingale measure (Kennedy et al., 2012) – i.e. we are already working in a risk-neutral, arbitrage-free setting.

Pricing Under SABR

Intuition. Hagan et al. (2002) used a perturbation (asymptotic) expansion to derive an approximate, closed-form implied volatility $\sigma_B(f,k)$. Plugging this into Black's formula prices European options *as if* volatility were constant at that level $\sigma_B(f,k)$.

Black's formula for a call option

$$V_{\text{call}} = D(t)\left(fN(d_1) - kN(d_2)\right), \qquad d_{1/2} = \frac{\ln(f/k) \pm \frac{1}{2}\sigma_B^2 t}{\sigma_B\sqrt{t}} \tag{3}$$

Symbols: $D(t)$ – discount factor; f – forward price; k – strike; $N(\cdot)$ – standard normal CDF; $\sigma_B(f,k)$ – the SABR-implied (Black) volatility.

This closed-form price is exactly why SABR became so popular: practitioners get fast prices and risk sensitivities (Vega, Vanna, Volga) without numerically solving the SDEs.

The Hagan (2002) Implied Volatility Formula I

Log-normal implied volatility expansion

$$\sigma_B(f,k) = \frac{\alpha}{(fk)^{\frac{1-\beta}{2}}\left\{1+\frac{(1-\beta)^2}{24}(\ln(f/k))^2+\frac{(1-\beta)^4}{1920}(\ln(f/k))^4+\ldots\right\}} \times \frac{z}{x(z)} \times \left\{1+\left[\frac{(1-\beta)^2}{24}\frac{\sigma^2}{(fk)^{1-\beta}}+\frac{1}{4}\frac{\rho\beta\nu\alpha}{(fk)^{(1-\beta)/2}}+\frac{2-3\rho^2}{24}\nu^2\right]t+\ldots\right\} \quad (4)$$

where

$$z = \frac{\nu}{\alpha}(fk)^{(1-\beta)/2}\ln(f/k), \qquad x(z) = \ln\left(\frac{\sqrt{1-2\rho z+z^2}+z-\rho}{1-\rho}\right) \quad (5)$$

The Hagan (2002) Implied Volatility Formula II

At-the-money special case ($f = k$):

ATM implied volatility

$$\sigma_{ATM} = \sigma_B(f,f) = \frac{\alpha}{f^{1-\beta}} \left\{ 1 + \left[\frac{(1-\beta)^2}{24} \frac{\alpha^2}{f^{2-2\beta}} + \frac{1}{4} \frac{\rho\beta\nu\alpha}{f^{1-\beta}} + \frac{2-3\rho^2}{24} \nu^2 \right] t + \ldots \right\} \quad (6)$$

Role of each parameter (intuition):

- β: main driver of skew steepness. When $\rho > 0$, decreasing β from 1 to 0 makes the skew slope more negative.
- ρ: a larger negative ρ also makes the skew slope further downward. ρ and β have comparable, hard-to-separate effects.
- ν (Volvol): controls the curvature of the smile – larger ν gives more curvature; longer maturities flatten it.
- α: sets the general level of ATM volatility.

In practice β is usually *fixed* (this chapter fixes $\beta = 0.5$), and $\{\alpha, \rho, \nu\}$ are calibrated to data.

The Normal Implied Volatility Expansion (Used in This Chapter)

Intuition. Swaption markets are often quoted in *normal* (basis-point) volatility rather than log-normal volatility, especially when rates can go negative. This chapter uses the normal-volatility expansion.

Normal implied volatility

$$\sigma_B(f,k) = \alpha \times \frac{z}{x(z)} \times A\big(1 + (B + C + D)t\big) \tag{7}$$

$$A = \frac{(1-\beta)(f-k)}{f^{(1-\beta)} - k^{(1-\beta)}}, \qquad B = \frac{-\beta(2-\beta)\alpha^2}{24(\sqrt{fk})^{(2-2\beta)}}$$

$$C = \frac{\rho\alpha\nu\beta}{4(\sqrt{fk})^{(1-\beta)}}, \qquad D = \frac{2-3\rho^2}{6}\frac{\nu^2}{4}, \qquad z = \frac{\nu}{\alpha}(k-f)$$

$$x(z) = \ln\left(\frac{\sqrt{1+2\rho z+z^2}+z+\rho}{1+\rho}\right) \tag{8}$$

Parameters to infer: $\{\alpha, \beta, \rho, \nu\}$, with $\alpha \in [0,\infty)$, $\beta \in [0,1]$, $\rho \in [-1,1]$, $\nu \in [0,\infty)$. This chapter fixes $\beta = 0.5$ (standard market practice) and calibrates $\{\alpha, \rho, \nu\}$ using Bayesian inference.

Python: The SABR Normal-Volatility Skew

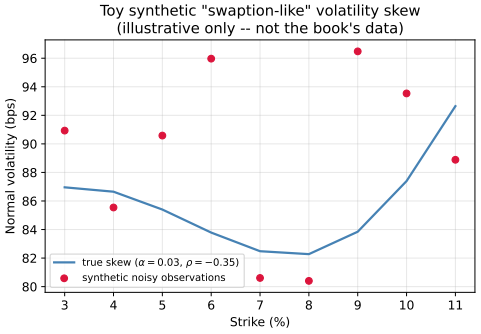
Implementing the normal-vol expansion directly from the formulas above lets us generate a full volatility skew for any candidate (α, ρ, ν) – this is the "forward model" whose parameters we will later infer with MCMC.

```python
import numpy as np

def sabr_normal_vol(k, alpha, rho, nu, beta=0.5, f=0.07, t=1.0):
    """Hagan et al. (2002) normal (basis-point) vol expansion."""
    k = np.asarray(k, dtype=float)
    fk_avg = np.sqrt(f * k)
    A = np.where(np.isclose(f, k), alpha * f**beta,
                 (1 - beta) * (f - k) / (f**(1-beta) - k**(1-beta)) * alpha)
    B = -beta*(2-beta)*alpha**2 / (24*fk_avg**(2-2*beta))
    C = rho*alpha*nu*beta / (4*fk_avg**(1-beta))
    D = (2 - 3*rho**2)/6.0 * (nu**2)/4.0
    z = (nu/alpha) * (k - f)
    xz = np.log((np.sqrt(1+2*rho*z+z**2)+z+rho)/(1+rho))
    zx = np.where(np.isclose(z, 0.0), 1.0, z/xz)
    sigma = A * zx * (1 + (B + C + D)*t)
    return sigma * 1e4          # convert to basis points

# Toy skew with alpha=0.03, rho=-0.35 (illustrative, NOT the book's data)
strikes = np.linspace(0.03, 0.11, 9)
vols_bps = sabr_normal_vol(strikes, alpha=0.03, rho=-0.35, nu=0.8)
```

Our Toy Running Example: A Synthetic "Swaption-Like" Skew

To make MALA, HMC and S2HMC concrete, we will use one toy inference problem throughout this chapter:

- Fix $\beta = 0.5$, $\nu = 0.8$, forward $f = 7\%$, maturity $t = 1$ (all illustrative, chosen by us – not the book's data).
- Set *true* parameters $\alpha = 0.03$, $\rho = -0.35$.
- Generate 9 synthetic strikes and add Gaussian noise (std 8 bps) to the true skew, mimicking noisy market quotes.
- **Goal:** recover the posterior over (α, ρ) from these 9 noisy points, using MALA and then HMC.



Notation and Background You Need I

You already know (Chapter 2) that Bayesian inference gives us a posterior $\pi(\mathbf{w}) \propto \text{likelihood}(\mathbf{w}) \times \text{prior}(\mathbf{w})$ over parameters $\mathbf{w}$, and that MCMC is one way to draw samples from it. Here is the vocabulary this chapter builds on, in plain language first:

- **Metropolis-Hastings (MH):** propose a random next step; accept it with a probability that keeps you exploring the posterior correctly, reject and stay put otherwise. This is the ancestor of every method in this chapter.
- **MALA:** Metropolis-Hastings where the proposal is *nudged* in the direction the posterior increases, using its gradient – for smarter proposals than a pure random walk.
- **HMC:** borrow physics – treat the negative log-posterior as a landscape, give your sample point a random "velocity" (momentum), and simulate it rolling over the landscape like a frictionless marble. This explores far more efficiently than a random walk because it can travel a long distance in a single proposal while still landing somewhere with high posterior probability.
- **Leapfrog integrator:** the numerical method used to simulate that rolling motion in small discrete time steps (since the equations of motion typically cannot be solved exactly).
- **Shadow Hamiltonian:** a slightly modified energy function that the leapfrog integrator conserves *almost* exactly – letting you take bigger, more efficient steps without paying for it in lots of rejections.

Notation and Background You Need II

Why go to all this trouble? A plain random-walk MH sampler explores the posterior in small, inefficient, highly-correlated steps – like a drunk person shuffling around a room. MALA gives it a compass (the gradient). HMC gives it a bicycle (momentum + physics) so it can cover the whole room efficiently. Shadow Hamiltonians (S2HMC) let that bicycle go faster without crashing (fewer rejections at large step sizes).

One important running thread: all these methods work on $\mathbf{w}$ = the SABR parameters $\{\alpha, \rho, \nu\}$ (or a subset, in our toy example just $\{\alpha, \rho\}$), and target the Bayesian posterior $\pi(\mathbf{w} \mid \text{swaption data})$.

Why MCMC for SABR Calibration?

Recall from Chapter 2: the SABR posterior over $\{\alpha, \rho, \nu\}$ given observed swaption volatilities is not available in closed form – we need a numerical method to sample from it.

Two families of options:

- **Variational inference** – approximate the posterior with a simpler, tractable distribution (fast, but approximate).
- **MCMC** – if you run it long enough, it is *guaranteed* to converge to the exact target posterior distribution.

This chapter implements MALA, HMC and S2HMC (all in PyTorch, per the book) to calibrate SABR to both simulated and ZAR market swaption data, and compares how well each recovers the parameters and predicts unseen data.

MALA: Intuition

Plain-language idea. Ordinary Metropolis-Hastings proposes a completely random next step – it has no sense of "uphill" or "downhill" on the posterior surface. This makes it a slow, wandering random walk in high dimensions.

MALA fixes this by using the **gradient** of the log-posterior to bias the proposal towards higher-density regions, then still uses a Metropolis-Hastings accept/reject step to correct for any bias the numerical discretisation introduces. Think of it as "a random walk with a compass that always points slightly uphill."

MALA: The Langevin Dynamics

Langevin diffusion (Eq. 3.17)

$$d\mathbf{w}_t = \tfrac{1}{2} f'(\mathbf{w})\, dt + dZ_t \tag{9}$$

Symbols: $\mathbf{w}$ – the parameters being sampled; $f'(\mathbf{w})$ – the first derivative (gradient) of the target log-density $f(\mathbf{w})$ with respect to $\mathbf{w}$; Z_t – a Brownian motion at fictitious time t.

Why we need an accept/reject step. This continuous-time SDE cannot usually be solved exactly. Discretising it (e.g. with the Euler-Maruyama scheme) introduces error that can break *detailed balance* – the property that guarantees the chain converges to the right target. A Metropolis acceptance step is added on top of the discretised proposal to restore detailed balance exactly.

Standard MALA proposal (discretised, step size ϵ):

$$\mathbf{w}^* \sim \mathcal{N}\big(\mathbf{w} + \tfrac{\epsilon}{2} f'(\mathbf{w}),\ \epsilon \mathbf{I}\big) \tag{10}$$

accepted with the usual Metropolis-Hastings probability $\min\big(1, \frac{\pi(\mathbf{w}^*) q(\mathbf{w}|\mathbf{w}^*)}{\pi(\mathbf{w}) q(\mathbf{w}^*|\mathbf{w})}\big)$, where q is the Gaussian proposal density above (this correction is what makes "MALA" the *Metropolis-Adjusted* Langevin Algorithm).

MALA: Properties

- Converges to the stationary (posterior) distribution *faster* than plain Metropolis-Hastings, because it exploits first-order gradient information.
- Samples produced are still **highly correlated** – each proposal is only a small, local nudge.
- Girolami & Calderhead (2011) extend MALA to use *second-order* gradient (curvature) information via a Riemannian manifold formulation – better mixing, at a higher computational cost.
- In this chapter, all three MCMC methods use a pre-conditioning matrix $\mathbf{M} = \mathbf{I}$ (identity), the typical default choice in practice.

Python: MALA on the Toy SABR Posterior (Log-Posterior)

We infer (α, ρ) from the 9 synthetic noisy swaption-like vols, with a Gaussian likelihood, weak Gaussian priors, and finite-difference gradients (from scratch, no autodiff):

```
def log_posterior(theta):                    # theta = (alpha, rho)
    alpha, rho = theta
    if alpha <= 1e-5 or abs(rho) >= 0.999: return -np.inf
    model = sabr_normal_vol(STRIKES, alpha, rho)
    ll = -0.5*np.sum(((OBS_VOLS - model)/OBS_NOISE_SD)**2)
    lp_a = -0.5*((alpha-0.02)/0.02)**2       # weak Gaussian prior on alpha
    lp_r = -0.5*((rho-0.0)/0.5)**2           # weak Gaussian prior on rho
    return ll + lp_a + lp_r

def grad_log_posterior(theta, h=1e-5):       # finite-difference gradient
    grad = np.zeros(2)
    for i in range(2):
        tp, tm = theta.copy(), theta.copy()
        tp[i] += h; tm[i] -= h
        grad[i] = (log_posterior(tp) - log_posterior(tm)) / (2*h)
    return grad
```

Python: MALA on the Toy SABR Posterior (Sampler Step)

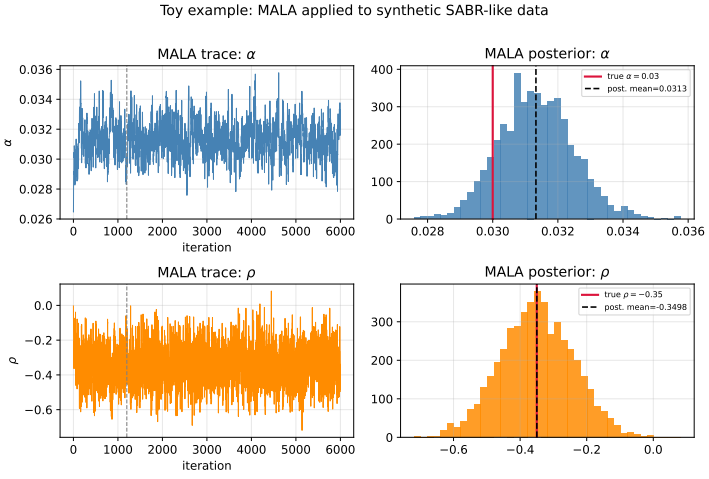
The MALA proposal, drift-adjusted using the gradient above, with the matching Metropolis-Hastings correction:

```python
def mala_step(theta, step_vec, rng):
    grad = grad_log_posterior(theta)
    mean_fwd = theta + 0.5*step_vec*grad
    prop = mean_fwd + np.sqrt(step_vec)*rng.standard_normal(2)
    grad_prop = grad_log_posterior(prop)
    mean_bwd = prop + 0.5*step_vec*grad_prop
    log_q_fwd = -np.sum((prop-mean_fwd)**2/step_vec)/2
    log_q_bwd = -np.sum((theta-mean_bwd)**2/step_vec)/2
    log_a = (log_posterior(prop)+log_q_bwd) - (log_posterior(theta)+log_q_fwd)
    accept = np.log(rng.uniform()) < log_a
    return (prop if accept else theta), accept
```

Illustrative toy result (our own simulation, not the book's data): running this for 6000 iterations (1200 burn-in) with a well-tuned step size gives an acceptance rate of about **68%** and recovers both true parameters closely.

Toy MALA Results

Illustrative only – our own toy simulation, not the book's data. Correlated trace plots reflect MALA's small, local proposal nudges; both posteriors nonetheless concentrate close to the true parameter values:



HMC: Intuition – Borrowing Physics

The key idea. Treat the negative log-posterior $U(\mathbf{w}) = -\log \pi(\mathbf{w})$ as a *potential energy landscape*. Give your current sample point $\mathbf{w}$ a random *momentum* $\mathbf{p}$ (like a push), then let it **roll** over the landscape following the laws of frictionless mechanics.

- Rolling downhill in U trades potential energy for kinetic energy – the point speeds up in low-U (high posterior density) regions.
- Because the total energy (Hamiltonian) is conserved, the point can travel *far* from where it started while remaining on a near-constant-probability contour – unlike a short-sighted random walk.
- This is why HMC proposals can be distant from the current point yet still be accepted with high probability – dramatically reducing autocorrelation between samples compared to MH or MALA.

HMC: Hamiltonian Dynamics

Hamiltonian dynamics (Eq. 3.18)

$$\frac{d\mathbf{w}}{dt} = \frac{\partial \mathrm{H}(\mathbf{w}, \mathbf{p})}{\partial \mathbf{p}}, \qquad \frac{d\mathbf{p}}{dt} = -\frac{\partial \mathrm{H}(\mathbf{w}, \mathbf{p})}{\partial \mathbf{w}} \tag{11}$$

Symbols: $\mathbf{w}$ – parameters ("position"); $\mathbf{p}$ – auxiliary momentum variable, same dimension as $\mathbf{w}$; $\mathrm{H}(\mathbf{w}, \mathbf{p})$ – the **Hamiltonian**, i.e. total energy $= U(\mathbf{w}) + K(\mathbf{p})$, where $U(\mathbf{w}) = -\log \pi(\mathbf{w})$ is the potential energy (from the target posterior) and $K(\mathbf{p})$ is the kinetic energy, typically a multivariate Gaussian quadratic form in $\mathbf{p}$.

HMC alternates two steps: (1) integrate these dynamics forward using the **leapfrog integrator**, then (2) apply a **Metropolis-Hastings correction** to account for the numerical integration error.

HMC: The Leapfrog Integrator

Why we need it. The Hamiltonian ODEs above generally cannot be solved in closed form, so we simulate them in discrete steps of size ϵ. The leapfrog scheme updates momentum, then position, then momentum again – a symmetric "half-step, full-step, half-step" pattern that is time-reversible and (approximately) energy-conserving.

Leapfrog update (Eq. 3.19)

$$\mathbf{p}_{t+\epsilon/2} = \mathbf{p}_t + \frac{\epsilon}{2}\frac{\partial \mathrm{H}(\mathbf{w}_t, \mathbf{p}_t)}{\partial \mathbf{w}} \tag{12}$$

$$\mathbf{w}_{t+\epsilon} = \mathbf{w}_t + \epsilon\, \mathbf{M}^{-1}\mathbf{p}_{t+\epsilon/2} \tag{13}$$

$$\mathbf{p}_{t+\epsilon} = \mathbf{p}_{t+\epsilon/2} + \frac{\epsilon}{2}\frac{\partial \mathrm{H}(\mathbf{w}_{t+\epsilon}, \mathbf{p}_{t+\epsilon/2})}{\partial \mathbf{w}} \tag{14}$$

Symbols: ϵ – discretisation step size; $\mathbf{M}$ – a covariance ("mass") matrix from the kinetic energy term.

Note: since $\mathrm{H} = U + K$ and $dp/dt = -\partial \mathrm{H}/\partial \mathbf{w} = -U'(\mathbf{w})$, the momentum half-step is equivalently written using the *gradient of the log-posterior* $\nabla \log \pi(\mathbf{w}) = -U'(\mathbf{w})$ as $\mathbf{p} \leftarrow \mathbf{p} + \frac{\epsilon}{2}\nabla \log \pi(\mathbf{w})$ – the form our Python code below uses.

HMC: Accept/Reject Step and Full Algorithm

Metropolis acceptance probability (Eq. 3.20)

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min\left(1, \frac{\exp(-\mathrm{H}(\mathbf{w}^*, \mathbf{p}^*))}{\exp(-\mathrm{H}(\mathbf{w}, \mathbf{p}))}\right) \tag{15}$$

If the leapfrog integrator solved the dynamics exactly, this would always accept (H perfectly conserved); the discretisation error is why this MH correction step is needed to keep the chain exact.

Algorithm 3.1: Hamiltonian Monte Carlo

Input: $N, \epsilon, L, w_{\text{init}}, \mathrm{H}(w, p)$. **Output:** $(w)_{m=0}^{N}$

1. $w_0 \leftarrow w_{\text{init}}$
2. **for** $m \to 1$ **to** N **do**
   - 3.1 $p_{m-1} \sim \mathcal{N}(0, \mathbf{M})$
   - 3.2 $p_m, w_m = \textsc{Leapfrog}(p_{m-1}, w_{m-1}, \epsilon, L, \mathrm{H})$ [Eq. 3.19]
   - 3.3 $\delta H = \mathrm{H}(w_{m-1}, p_{m-1}) - \mathrm{H}(w_m, p_m)$
   - 3.4 $\alpha_m = \min(1, \exp(\delta H))$; $u_m \sim \mathsf{Unif}(0, 1)$
   - 3.5 $w_m = \textsc{Metropolis}(\alpha_m, u_m, w_m, w_{m-1})$ [Eq. 3.20]
3. **end for**

HMC Parameters and the MALA Connection

- **Step size ϵ:** typically tuned to hit a target acceptance rate via the *dual averaging* method (Hoffman & Gelman, 2014).
- **Trajectory length L:** the number of leapfrog steps per proposal – a harder parameter to tune well.
- **Neat fact:** when $L = 1$, HMC reduces exactly to MALA! HMC is thus a strict generalisation of MALA that lets the marble roll for longer before we check whether to accept.
- The overall sampler is a Gibbs scheme: resample momentum $\mathbf{p}$, then update $\mathbf{w}$ given that momentum, repeat.

Worked Example: One Full HMC Iteration, Step by Step

Toy target: sample from a standard Gaussian posterior $\pi(w) \propto \exp(-w^2/2)$, so $U(w) = w^2/2$, $U'(w) = w$. Take step size $\epsilon = 0.1$.

Step 1 – Resample momentum. Draw $p_0 \sim \mathcal{N}(0, 1)$. Suppose we draw $p_0 = 0.3$, and our current sample is $w_0 = 1.0$.

Step 2 – Run L leapfrog steps. First half-step of momentum:

$$p_{1/2} = p_0 - \tfrac{\epsilon}{2} U'(w_0) = 0.3 - 0.05(1.0) = 0.25$$

Full position step:

$$w_1 = w_0 + \epsilon\, p_{1/2} = 1.0 + 0.1(0.25) = 1.025$$

Second half-step of momentum:

$$p_1 = p_{1/2} - \tfrac{\epsilon}{2} U'(w_1) = 0.25 - 0.05(1.025) = 0.19875$$

(This is repeated L times in total; here we show the first of L steps.)

Worked Example (continued): Acceptance

Step 3 – Compute the acceptance probability. After the leapfrog step above,

$$H_0 = U(w_0) + K(p_0) = \tfrac{1}{2}(1.0)^2 + \tfrac{1}{2}(0.3)^2 = 0.545000$$
$$H_1 = U(w_1) + K(p_1) = \tfrac{1}{2}(1.025)^2 + \tfrac{1}{2}(0.19875)^2 = 0.545063$$

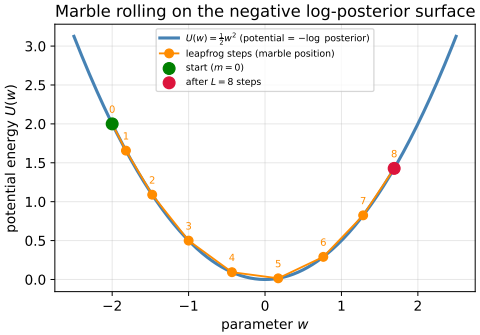
$$\mathbb{P}(\text{accept}) = \min\big(1, \exp(H_0 - H_1)\big) = \min\big(1, \exp(-0.000063)\big) \approx 0.99994$$

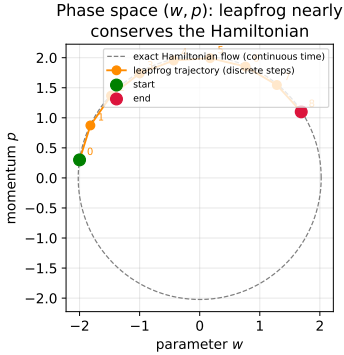
Step 4 – Accept or reject. Draw $u \sim \text{Unif}(0, 1)$. Since the acceptance probability is ≈ 1 here (the leapfrog step nearly conserved energy exactly, as expected for a small ϵ on a simple quadratic potential), we almost always accept: $w_{\text{new}} = w_1 = 1.025$.

Takeaway: because H is nearly conserved along the whole trajectory, HMC can run L such steps (potentially moving quite far from w_0) while keeping the acceptance probability close to 1 – this is the source of its efficiency advantage over a plain random walk or MALA.

Visualising the Leapfrog Trajectory

Schematic: one HMC trajectory built from L leapfrog steps





Left: the marble (orange dots, numbered by leapfrog step) rolls down and back up the potential well $U(w) = \frac{1}{2}w^2$. **Right:** in phase space (w, p), the exact continuous-time orbit is a perfect ellipse (dashed grey); the discrete leapfrog steps (orange) stay very close to it – this is exactly why the acceptance probability stays near 1.

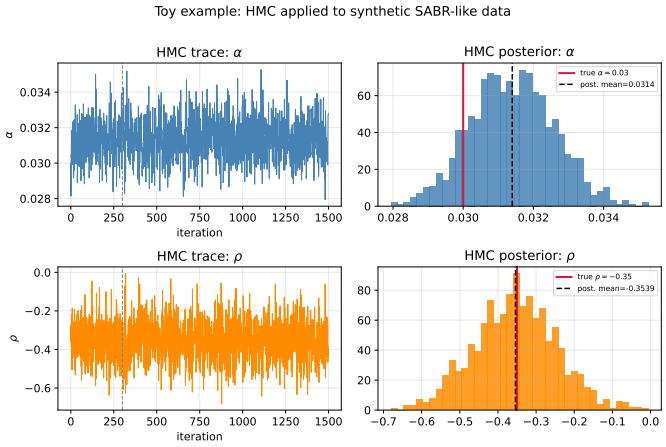
Python: HMC on the Toy SABR Posterior

Applying HMC to the same toy (α, ρ) inference problem as MALA, using a diagonal mass matrix (to handle α and ρ living on very different natural scales – $\alpha \sim 0.01$ vs. $\rho \sim 1$):

```python
def leapfrog(w, p, eps, L, precond):
    grad = grad_log_posterior(w)
    for _ in range(L):
        p = p + 0.5*eps*grad
        w = w + eps*precond*p
        grad = grad_log_posterior(w)
        p = p + 0.5*eps*grad
    return w, p

def hmc_step(theta, eps, L, precond, mass, rng):
    p0 = rng.standard_normal(2) * np.sqrt(mass)
    w, p = leapfrog(theta.copy(), p0.copy(), eps, L, precond)
    H0 = -log_posterior(theta) + 0.5*np.sum(p0**2*precond)
    H1 = -log_posterior(w)     + 0.5*np.sum(p**2 *precond)
    accept = np.log(rng.uniform()) < (H0 - H1)
    return (w if accept else theta), accept
# Book's setting: L=25 leapfrog steps, step size tuned via dual averaging
# to target an 80% acceptance rate (mirrored in our toy demo below).
```

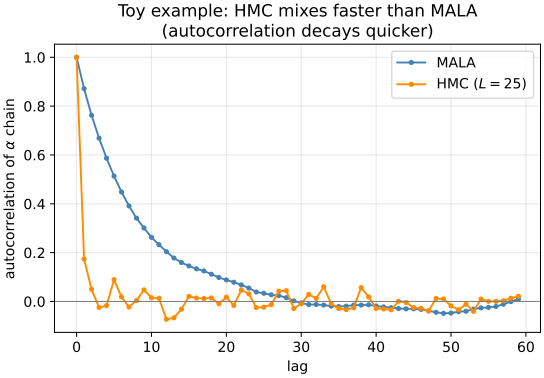
Toy HMC Results

Illustrative only. Running HMC with $L = 25$ leapfrog steps for 1500 iterations (300 burn-in) gives an acceptance rate of about **96%** and again recovers both true parameters:



MALA vs. HMC: Why HMC Mixes Faster

Comparing the autocorrelation of the α chain from our toy MALA and HMC runs makes the payoff of "rolling like a marble" concrete:



With $L = 25$ leapfrog steps per proposal, HMC's samples decorrelate almost immediately, while MALA's small local nudges leave noticeable correlation out to lag $\sim$20 – more effective samples per iteration.

S2HMC: Intuition

The remaining problem with HMC. Even HMC still produces autocorrelated samples. Why? The leapfrog integrator only conserves the *true* Hamiltonian up to second order, $\mathcal{O}(\epsilon^2)$. Over L steps this error accumulates, so $\delta H = H(\mathbf{w}_{m-1}, \mathbf{p}_{m-1}) - H(\mathbf{w}_m, \mathbf{p}_m)$ can become large – causing more rejections in the Metropolis step.

Two obvious fixes – use a higher-order (more accurate) integrator, or shrink ϵ and L – both cost more compute time.

A cleverer fix. Run a backward error analysis on the leapfrog integrator to find a *modified* ("shadow") Hamiltonian that the leapfrog integrator conserves *almost exactly* – much better than it conserves the true Hamiltonian. Sample using the shadow Hamiltonian instead, then correct for the (small) difference so we still target the true posterior.

The Shadow Hamiltonian Expansion

Shadow Hamiltonian as an asymptotic expansion (Eq. 3.21–3.22)

$$\tilde{\mathrm{H}} = \mathrm{H} + \epsilon \mathrm{H}_2 + \epsilon^2 \mathrm{H}_3 + \epsilon^3 \mathrm{H}_4 + \dots \tag{16}$$

$$\tilde{\mathrm{H}}^{[k]} = \mathrm{H} + \epsilon \mathrm{H}_2 + \epsilon^2 \mathrm{H}_3 + \epsilon^3 \mathrm{H}_4 + \dots = \tilde{\mathrm{H}} + \mathcal{O}(\epsilon^k) \tag{17}$$

Symbols: $\tilde{\mathrm{H}}$ – the (infinite-series) shadow Hamiltonian; $\tilde{\mathrm{H}}^{[k]}$ – a k-th order truncation used in practice to keep the expansion from diverging. By comparing Taylor coefficients between the true flow and the leapfrog-generated flow, the correction terms H_k can be derived explicitly. For symplectic (structure-preserving) integrators like leapfrog, these modified equations can themselves be treated as Hamiltonian.

Fourth-Order Shadow Hamiltonian

This chapter uses a **fourth-order truncation** $\tilde{\mathrm{H}}^{[4]}$, which is conserved with accuracy $\mathcal{O}(\epsilon^4)$ by the leapfrog integrator – two orders better than the true Hamiltonian's $\mathcal{O}(\epsilon^2)$.

Fourth-order shadow Hamiltonian (Eq. 3.23)

$$\tilde{\mathrm{H}}^{[4]} = U(\mathbf{w}) + K(\mathbf{p}) + \frac{\epsilon^2}{12}\mathrm{K}_{\mathbf{p}}^T\mathrm{U}_{\mathbf{ww}}\mathrm{K}_{\mathbf{p}} - \frac{\epsilon^2}{24}\mathrm{U}_{\mathbf{w}}^T\mathrm{K}_{\mathbf{pp}}\mathrm{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4) \tag{18}$$

Symbols: $\mathrm{U}_{\mathbf{w}}$, $\mathrm{K}_{\mathbf{p}}$ – Jacobians (gradients) of the potential/kinetic energy; $\mathrm{U}_{\mathbf{ww}}$, $\mathrm{K}_{\mathbf{pp}}$ – their Hessians.

Problem: this $\tilde{\mathrm{H}}^{[4]}$ mixes $\mathbf{w}$ and $\mathbf{p}$ together (it is *non-separable*) – expensive to work with, requiring tuning of extra momentum-acceptance criteria.

Making It Separable: S2HMC

The fix (Sweet et al., 2009). Pre-process the parameter and momentum vectors, *before* running the ordinary leapfrog integrator, so that the resulting shadow Hamiltonian becomes **separable** again (a clean sum of a potential term and a kinetic term).

Separable shadow Hamiltonian (Eq. 3.24)

$$\tilde{\mathrm{H}}(\mathbf{w}, \mathbf{p}) = U(\mathbf{w}) + K(\mathbf{p}) + \frac{\epsilon^2}{24} \mathrm{U}_{\mathbf{w}}^T \mathbf{M}^{-1} \mathrm{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4) \tag{19}$$

Positions and momenta are propagated on this shadow Hamiltonian after a **reversible mapping** $(\hat{\mathbf{w}}, \hat{\mathbf{p}}) = \mathcal{X}(\mathbf{w}, \mathbf{p})$, found via fixed-point iteration:

Reversible mapping (Eq. 3.25–3.26)

$$\hat{\mathbf{p}} = \mathbf{p} - \frac{\epsilon}{24} \Big[\mathrm{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) - \mathrm{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \Big] \tag{20}$$

$$\hat{\mathbf{w}} = \mathbf{w} + \frac{\epsilon^2 \mathbf{M}^{-1}}{24} \Big[\mathrm{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) + \mathrm{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \Big] \tag{21}$$

S2HMC: Post-Processing and Importance Weights

After leapfrog is run on $(\hat{\mathbf{w}}, \hat{\mathbf{p}})$, the mapping is **reversed** using another fixed-point iteration:

Reverse mapping (Eq. 3.27–3.28)

$$\mathbf{w} = \hat{\mathbf{w}} - \frac{\epsilon^2 \mathbf{M}^{-1}}{24}\Big[\mathrm{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1}\hat{\mathbf{p}}) + \mathrm{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1}\hat{\mathbf{p}})\Big] \tag{22}$$

$$\mathbf{p} = \hat{\mathbf{p}} + \frac{\epsilon}{24}\Big[\mathrm{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1}\hat{\mathbf{p}}) - \mathrm{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1}\hat{\mathbf{p}})\Big] \tag{23}$$

Importance weights. Because we actually sampled from the shadow density (not the true one), *importance weights* are computed after sampling – based on the difference between the true and shadow Hamiltonians – to correct back to the true target posterior.

Neat fact: S2HMC *degenerates exactly to HMC* when the shadow Hamiltonian is set equal to the true Hamiltonian – so S2HMC is again a strict generalisation, this time of HMC.

Python: Sketch of the S2HMC Reversible Mapping

This is a structural sketch of the fixed-point solve in Eqs. 3.25–3.26 (illustrative – not a full production implementation):

```python
def s2hmc_forward_map(w, p, eps, Minv, grad_U, n_fp=6):
    """Fixed-point iteration for the reversible mapping (w,p) -> (what, phat)."""
    what, phat = w.copy(), p.copy()
    for _ in range(n_fp):                        # iterate to convergence
        gplus  = grad_U(w + eps*Minv*phat)
        gminus = grad_U(w - eps*Minv*phat)
        phat = p - (eps/24.0) * (gplus - gminus)
        what = w + (eps**2 * Minv/24.0) * (gplus + gminus)
    return what, phat

# Usage inside one S2HMC iteration:
#   1. what, phat = s2hmc_forward_map(w, p, eps, Minv, grad_U)
#   2. what, phat = leapfrog(what, phat, eps, L, Minv)      # ordinary leapfrog
#   3. w, p       = s2hmc_reverse_map(what, phat, eps, Minv, grad_U)
#   4. accept/reject using the SHADOW Hamiltonian, then reweight samples
#      by the true/shadow density ratio (importance weights).
```

Comparing MALA, HMC and S2HMC

	MALA	HMC	S2HMC
Uses gradient info?	Yes (1st order)	Yes	Yes
Simulated dynamics	Langevin (1 step)	Hamiltonian, L steps	Hamiltonian, L steps
Integrator	Euler-Maruyama	Leapfrog	Pre/post-processed leapfrog
Energy conserved to	–	$\mathcal{O}(\epsilon^2)$	$\mathcal{O}(\epsilon^4)$
Extra correction	MH accept/reject	MH accept/reject	MH accept/reject + importance weights
Special case of	–	MALA when $L = 1$	HMC when shadow $=$ true H

Book's finding (previewed): S2HMC tends to allow larger, more efficient steps (fewer rejections) at the cost of somewhat higher posterior standard deviations in this chapter's experiments – more in Section 3.5.

Data Used in This Chapter

Two datasets are used to calibrate and compare MALA, HMC and S2HMC:

- **Simulated data:** generated directly from the SABR model with known true parameters $\{\alpha = 0.0130,\ \rho = -0.23,\ \nu = 1.21\}$ for the **1W into 1Y** swaption. Because the truth is known here, this dataset checks whether each MCMC method can *recover* the correct parameters.
- **Real-world data:** South African Rand (ZAR) swaption market data, for the **6M into 1Y** swaption, as observed on **28 June 2024**, extracted from a calibrated model from a market data provider.

Using both lets the chapter validate correctness (simulated data, known truth) and real-world applicability (ZAR market data) side by side.

Performance Metrics Used

Predictive performance on **unseen** data is assessed using four standard regression metrics:

Metrics

$$\text{MSE} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \qquad \text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

$$\text{MAPE} = \frac{1}{n}\sum_i \left|\frac{y_i - \hat{y}_i}{y_i}\right| \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

where y_i is the observed volatility, $\hat{y}_i$ the model-predicted volatility, and $\bar{y}$ the sample mean.

Experimental setup (as reported): five chains of 2000 samples per method, with a burn-in of 1000; $\mathbf{M} = \mathbf{I}$ throughout; 25 leapfrog/integration steps for HMC and S2HMC; step sizes tuned via dual averaging targeting a 60% acceptance rate for MALA and 80% for HMC/S2HMC.

Calibration on Simulated Data (Table 3.1)

The book reports that MALA, HMC_{25} and S2HMC_{25} all reproduce the parameters that generated the simulated 1W into 1Y swaption data closely:

Parameter	True	MALA	HMC_{25}	S2HMC_{25}
α	0.0130	0.0129 (0.0191)	0.0129 (0.0264)	0.0130 (0.0277)
ρ	-0.23	-0.2301 (0.0092)	-0.2302 (0.0106)	-0.2303 (0.0104)
ν	1.21	1.2147 (0.0098)	1.2116 (0.0131)	1.2106 (0.0136)

(Values in parentheses are posterior standard deviations.)

The book reports that **S2HMC produces the highest standard deviations** for each parameter, while **MALA has the lowest** – i.e. MALA's posterior looks the most confident, though (as we saw with autocorrelation) it explores less efficiently per iteration.

Calibration on ZAR Market Data (Table 3.2)

Results for the real 6M into 1Y ZAR swaption data (28 June 2024):

Parameter	MALA	HMC_{25}	S2HMC_{25}
α	0.0347 (0.0027)	0.0347 (0.0029)	0.0347 (0.0031)
ρ	0.6004 (0.0043)	0.6001 (0.0044)	0.6004 (0.0045)
ν	0.9176 (0.0044)	0.9182 (0.0051)	0.9183 (0.0053)

The book reports that all three methods **agree closely** on the calibrated parameters, and again **S2HMC produces the largest standard deviations while MALA produces the lowest** – consistent with the simulated-data findings, suggesting this is a general pattern rather than a coincidence of one dataset.

Predictive Performance on Unseen Data (Tables 3.3–3.4)

Simulated 1W into 1Y (unseen) data:

Metric	MALA	HMC_{25}	S2HMC_{25}
R^2	0.9999992851	0.999999230	0.999999659
MSE	0.0007668415	0.000825566	**0.000365645**
MAPE	0.0004271379	0.00046407	**0.0003041687**
MAE	0.0220524944	0.023652592	**0.015996114**

ZAR 6M into 1Y (unseen) market data:

Metric	MALA	HMC_{25}	S2HMC_{25}
R^2	0.9999933461	0.9999932867	**0.9999934771**
MSE	0.0124373011	0.0125490858	**0.0121925901**
MAPE	0.0007746001	**0.0007267914**	0.0007388930
MAE	0.0882670898	**0.0854889729**	0.0856298624

The book reports that all methods perform similarly on unseen data, with **HMC and S2HMC displaying the best performance** across the various metrics.

Qualitative Findings: Fitting the Skew

The book reports:

- On the **simulated** 1W into 1Y skew, all three methods fit the data well, with predictions **most confident near the at-the-money (ATM) point** and progressively less confident towards the wings of the skew – and **less confident on the right wing than the left wing**.
- On the **ZAR market-observed** 6M into 1Y skew, the same general pattern holds, except the methods are confident in their predictions on the **left wing** specifically.
- Negative log-likelihood convergence traces (tracked across samples) show that **all three methods converge** on both the simulated and ZAR market-observed data.

Intuition: confidence bands are narrow where there is the most "pull" from data (near the liquid ATM point) and widen out where the model is extrapolating further from where α is best identified.

Qualitative Findings: Posterior Shapes

The book reports that, on both the simulated and ZAR market-observed swaption data, the three MCMC approaches **agree on the shape** of the marginal posterior distributions for α, ρ and ν.

Why this matters in practice:

- Practitioners can use these posterior distributions to understand the *range* of plausible parameter values, not just a single point estimate – something conventional calibration techniques (like maximum-likelihood fitting) do not directly provide.
- This makes it easier to construct realistic **stress scenarios** for the SABR parameters, relevant for e.g. economic capital calculations that depend on how the volatility skew could move.

Chapter Summary

- Fitting the volatility skew matters for practitioners trading non-linear interest-rate derivatives, and **SABR** is the standard model used in that market.
- This chapter presented a (book-reported) **first-in-literature probabilistic framework** to infer the volatility skew in interest-rate markets using SABR, calibrated on both simulated and ZAR-observed swaption data.
- Three MCMC samplers were built up from first principles: **MALA** (gradient-nudged random walk), **HMC** (physics-based, momentum + leapfrog), and **S2HMC** (shadow-Hamiltonian refinement of HMC).
- **The book reports:** all three methods reproduce the observed skew on both datasets and perform similarly on unseen data, with **S2HMC showing marginal outperformance**. Confidence bands are tightest near the at-the-money point and widen towards the wings.

Limitations and What's Next

The book notes this work could be extended by:

- Considering the **entire** interest-rate volatility surface, rather than just the single 1W into 1Y and 6M into 1Y swaption slices used here.
- Comparing SABR against **other models** that can also capture the volatility skew.

Looking ahead: the next chapter turns to learning the *entire* volatility surface using **sparse Gaussian process models**, applied to developed and emerging equity option markets – building directly on the Bayesian/MCMC machinery established here.

Key Takeaways

1. **SABR** models volatility itself as a random process, correlated with the asset price – fixing the shortcomings of constant- and local-volatility models for capturing the smile/skew.
2. **MH** $\rightarrow$ **MALA** $\rightarrow$ **HMC** $\rightarrow$ **S2HMC** form a natural progression: each adds a smarter way of proposing the next MCMC state, trading extra computation for faster mixing and lower autocorrelation.
3. **HMC's core trick** is treating sampling as physics: momentum plus a (nearly) energy-conserving leapfrog integrator lets proposals travel far while staying likely to be accepted.
4. **S2HMC** squeezes out further efficiency by having the integrator conserve a modified (*shadow*) Hamiltonian almost exactly, correcting back to the true posterior via importance weights.
5. These same MCMC building blocks reappear throughout the rest of this book – make sure they are solid before moving on.